

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 3

Implementation of Convolutional Neural Networks (CNNs) for Image Classification

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026– 27

1. Objective

To understand the working principle of Convolutional Neural Networks by implementing convolution, pooling, feature map visualization, and image classification using TensorFlow/Keras.

2. Learning Outcomes

Students will be able to

1. Explain the convolution operation.
2. Compute output dimensions using stride and padding.
3. Visualize feature maps.
4. Implement max pooling and average pooling.
5. Calculate CNN parameters.
6. Build a CNN for image classification.
7. Evaluate CNN performance.

3. Background Theory

A Convolutional Neural Network consists of

- Convolution Layer
- Activation Layer
- Pooling Layer
- Fully Connected Layer

The convolution operation is

$$Y(i, j) = \sum_m \sum_n X(i + m, j + n) K(m, n)$$

where

- X : Input Image
- K : Kernel (Filter)
- Y : Feature Map

The output feature map size is

$$\frac{N - F + 2P}{S} + 1$$

where

N Input Size

F Filter Size

P Padding

S Stride

3.1 Common Convolution Kernels

A **kernel** (or **filter**) is a small matrix that slides over an input image to extract important visual features such as edges, textures, corners, and smooth regions. During convolution, the kernel is multiplied element-wise with the corresponding image pixels, and the results are summed to produce a single value in the output feature map. Different kernels highlight different image characteristics.

1. Identity Kernel

The identity kernel preserves the original image without modifying its pixel values.

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

Purpose:

- Preserves the original image.
- Used for understanding the convolution operation.

2. Sobel-X Kernel (Vertical Edge Detection)

The Sobel-X kernel detects **vertical edges** by measuring intensity changes along the horizontal direction.

$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

Applications:

- Object detection
- Lane detection
- Face recognition

3. Sobel-Y Kernel (Horizontal Edge Detection)

The Sobel-Y kernel detects **horizontal edges** by measuring intensity changes along the vertical direction.

$$\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

Applications:

- Text detection
- Boundary extraction
- Medical image analysis

4. Laplacian Kernel

The Laplacian kernel detects edges in all directions using the second-order derivative of image intensity.

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Applications:

- Edge enhancement
- Satellite image analysis
- Medical imaging

5. Sharpening Kernel

This kernel enhances fine details by emphasizing high-frequency components.

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Applications:

- Image enhancement
- Optical Character Recognition (OCR)
- Face recognition

6. Box Blur Kernel

The Box Blur kernel smooths an image by replacing each pixel with the average of its neighboring pixels.

$$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Applications:

- Noise removal
- Image smoothing
- Image preprocessing

7. Gaussian Blur Kernel

Gaussian blur smooths the image while preserving the overall structure better than a simple average filter.

$$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

Applications:

- Noise reduction
- Computer vision preprocessing
- Feature extraction

8. Emboss Kernel

The Emboss kernel creates a three-dimensional appearance by emphasizing directional changes in intensity.

$$\begin{bmatrix} -2 & -1 & 0 \\ -1 & 1 & 1 \\ 0 & 1 & 2 \end{bmatrix}$$

Applications:

- Texture analysis
- Artistic image effects

9. Outline Detection Kernel

This kernel highlights object boundaries by emphasizing regions with rapid intensity changes.

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

Applications:

- Image segmentation
- Boundary detection
- Shape extraction

10. Motion Blur Kernel

This kernel simulates motion blur along the diagonal direction.

$$\frac{1}{5} \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

Applications:

- Motion simulation
- Image restoration
- Video processing

Summary of Common Kernels

Kernel	Size	Purpose	Applications
Identity	3×3	Preserve original image	Baseline comparison
Sobel-X	3×3	Detect vertical edges	Object detection
Sobel-Y	3×3	Detect horizontal edges	Boundary detection
Laplacian	3×3	Detect edges in all directions	Medical imaging
Sharpen	3×3	Enhance details	Image enhancement
Box Blur	3×3	Smooth image	Noise removal
Gaussian Blur	3×3	Reduce noise	Computer vision
Emboss	3×3	3D effect	Artistic filtering
Outline	3×3	Boundary extraction	Segmentation
Motion Blur	5×5	Simulate motion	Video processing

4. Numerical Example 1: Convolution

Consider the following input image.

$$X = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

Kernel

$$K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Output

First position

$$1(1) + 2(0) + 4(0) + 5(1) = 6$$

Second position

$$2(1) + 3(0) + 5(0) + 6(1) = 8$$

Third position

$$4(1) + 5(0) + 7(0) + 8(1) = 12$$

Fourth position

$$5(1) + 6(0) + 8(0) + 9(1) = 14$$

Feature Map

$$\begin{bmatrix} 6 & 8 \\ 12 & 14 \end{bmatrix}$$

Verified Output (TensorFlow)

Running the same convolution in TensorFlow/Keras with `tf.nn.conv2d`:

Ex1 - Convolution feature map:

```
[[ 6.  8.]  
 [12. 14.]]
```

The numerical output matches the hand-computed feature map exactly.

5. Numerical Example 2: Max Pooling

Feature map

$$\begin{bmatrix} 1 & 5 & 2 & 3 \\ 7 & 8 & 1 & 0 \\ 4 & 6 & 9 & 5 \\ 2 & 3 & 1 & 8 \end{bmatrix}$$

Using a 2×2 max pooling filter,
Window 1

$$\begin{bmatrix} 1 & 5 \\ 7 & 8 \end{bmatrix} \rightarrow 8$$

Window 2

$$\begin{bmatrix} 2 & 3 \\ 1 & 0 \end{bmatrix} \rightarrow 3$$

Window 3

$$\begin{bmatrix} 4 & 6 \\ 2 & 3 \end{bmatrix} \rightarrow 6$$

Window 4

$$\begin{bmatrix} 9 & 5 \\ 1 & 8 \end{bmatrix} \rightarrow 9$$

Output

$$\begin{bmatrix} 8 & 3 \\ 6 & 9 \end{bmatrix}$$

Verified Output (TensorFlow)

Ex2 - Max-pooling output:

```
[[8. 3.]  
 [6. 9.]]
```

The numerical output matches the hand-computed result.

6. Numerical Example 3: Parameter Calculation

Suppose

Input Image

$$32 \times 32 \times 3$$

Convolution Layer

- Filters = 16
- Kernel = 3×3

Parameters

$$(3 \times 3 \times 3 + 1) \times 16$$

$$= (27 + 1) \times 16$$

$$= 448$$

Therefore,

Total trainable parameters = 448.

Verified Output (TensorFlow)

Ex3 - Trainable params (16 filters, 3x3, RGB): 448

The layer `Conv2D(16, (3,3), input_shape=(32,32,3))` reports exactly 448 trainable parameters, confirming the calculation.

7. Dataset

Dataset: CIFAR-10

- Training Images : 50,000
- Testing Images : 10,000
- Classes : 10
- Image Size : $32 \times 32 \times 3$

Verified Dataset Shapes

Train: (50000, 32, 32, 3) Test: (10000, 32, 32, 3) Classes: 10

8. Experimental Procedure

Task 1

Load CIFAR-10.

- Display ten sample images.
- Print dataset dimensions.
- Plot class distribution.

Task 2

Implement a convolution layer.

Compare

- Kernel = 3×3
- Kernel = 5×5
- Kernel = 7×7

Record feature map sizes.

Task 3

Study hyperparameters.

Compare

- Stride = 1
- Stride = 2
- Padding = Same
- Padding = Valid

Compute output dimensions.

Task 4

Visualize feature maps after the first convolution layer.

Display at least eight feature maps.

Task 5

Compare

- Max Pooling
- Average Pooling

Compare output size and accuracy.

Task 6

Construct the following CNN.

Input $\rightarrow$ *Conv* $\rightarrow$ *ReLU* $\rightarrow$ *MaxPool* $\rightarrow$ *Conv* $\rightarrow$ *ReLU* $\rightarrow$ *MaxPool* $\rightarrow$ *Flatten* $\rightarrow$ *Dense* $\rightarrow$ *Softmax*

Train for

- Optimizer : Adam
- Maximum training budget : 20 epochs
- Batch Size : 32
- Early stopping : enabled, patience = 5 (restore best weights)

Task 7

Evaluate

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

9. Mandatory Plots

Generate

1. Sample Images
2. Class Distribution
3. Training Accuracy
4. Validation Accuracy
5. Training Loss
6. Validation Loss
7. Feature Maps

8. Confusion Matrix

Write a 2–3 line inference for each plot.

Plot 1: Sample CIFAR-10 Images

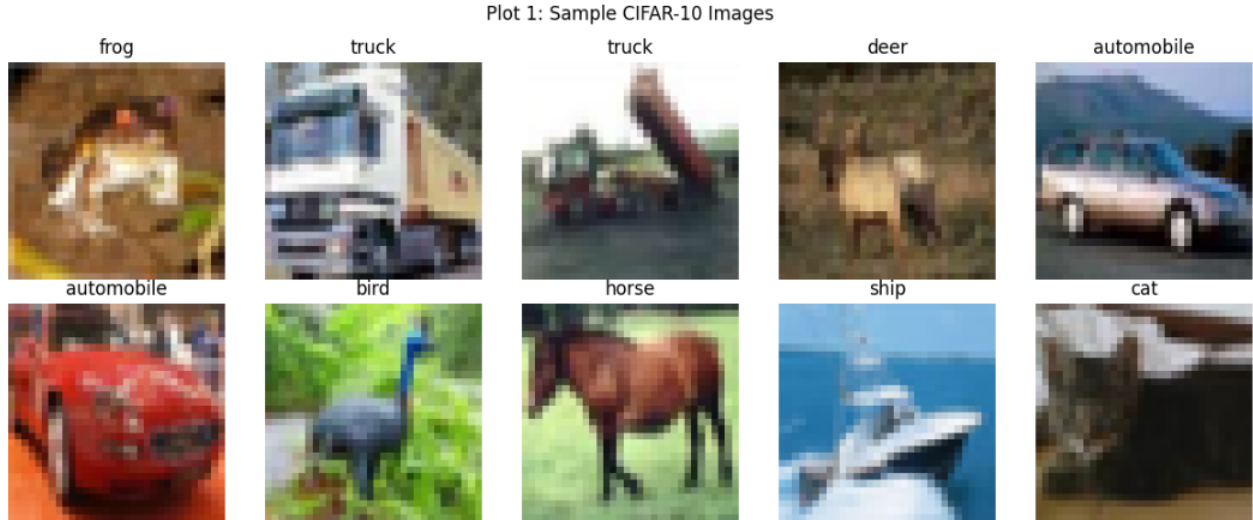


Figure 1: Ten random sample images from the CIFAR-10 training set (one per class).

Inference: The ten samples show clear representatives of each of the ten classes (airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck). Images are low resolution (32×32) with varied backgrounds and poses, indicating that the model must learn robust, viewpoint-invariant features. Visual similarity between certain classes (e.g., cat vs. dog, deer vs. horse) foreshadows where misclassifications are likely.

Plot 2: Class Distribution (Train)

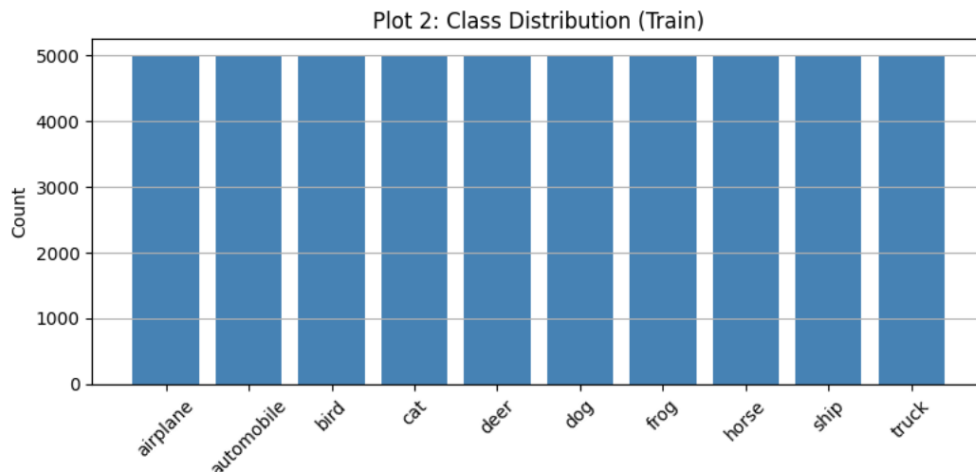


Figure 2: Distribution of the ten CIFAR-10 classes in the training set.

Inference: All ten classes contain exactly 5000 training samples each, confirming that CIFAR-10 is perfectly balanced. Because of this balance, plain accuracy is a reliable metric and no resampling or

class weighting is required. A balanced dataset also ensures that precision, recall, and F1-score will be comparable across classes.

Plot 3: Effect of Kernel Size on Feature Maps

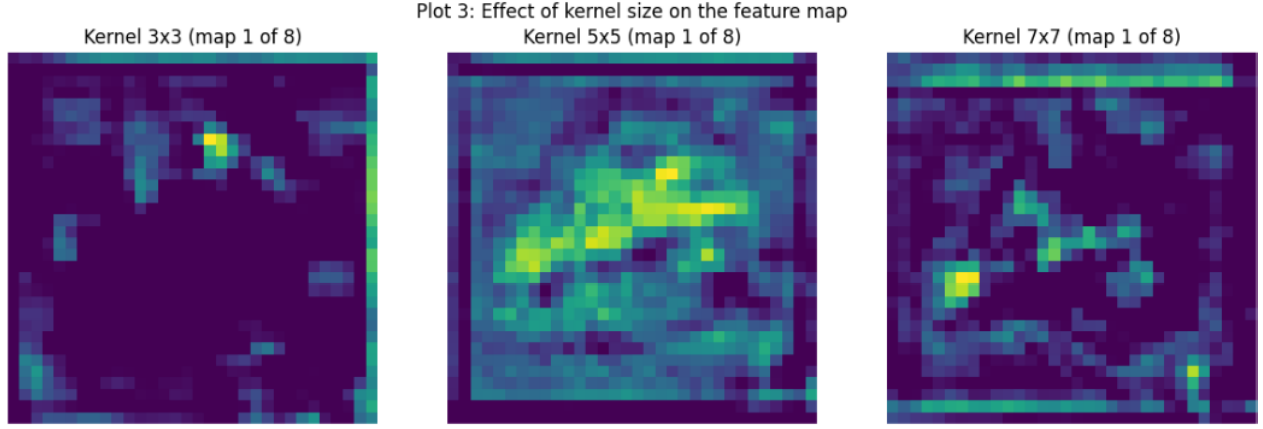


Figure 3: First feature map produced by 3×3 , 5×5 , and 7×7 kernels with `padding='same'`.

Inference: With `padding='same'` and stride 1, all three kernels keep the spatial size at 32×32 , so the input resolution is preserved. Larger kernels (7×7) capture broader contextual patterns but require more parameters and computation. Smaller kernels (3×3) capture finer local textures and are the standard choice in modern CNNs (e.g., VGG).

Plot 4: Effect of Stride and Padding on Feature Map Size (Task 3)

This experiment measures the output feature-map size for the input image 32×32 using a 3×3 kernel, varying the stride S and the padding P , and applying the standard formula

$$N_{\text{out}} = \left\lfloor \frac{N - F + 2P}{S} \right\rfloor + 1.$$

Configuration	Output Size	Downsampling?
Stride = 1, Padding = Same	32×32	No (spatial size preserved)
Stride = 2, Padding = Same	16×16	Yes ($2\times$ reduction)
Stride = 1, Padding = Valid	30×30	No, but border shrinks by $F - 1$
Stride = 2, Padding = Valid	15×15	Yes ($2\times$ reduction, plus border loss)

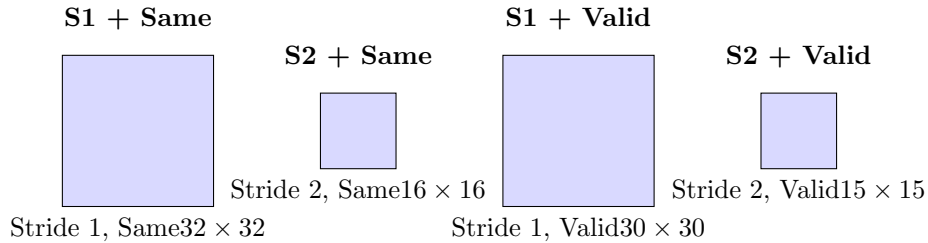


Figure 4: Plot 4: Feature-map sizes for a 32×32 input with a 3×3 kernel across the four stride/padding configurations. Panel size is proportional to the output resolution.

Inference: Stride 2 halves the spatial dimensions and therefore performs aggressive downsampling, which reduces computation and increases the effective receptive field. *Same* padding preserves the input spatial size when stride is 1, whereas *Valid* padding shrinks the feature-map by $F - 1$ pixels per dimension because no border zeros are added. Combining stride 2 with valid padding produces the smallest feature map (15×15), i.e., the strongest compression.

Plot 5: Eight Feature Maps after First Conv2D Layer

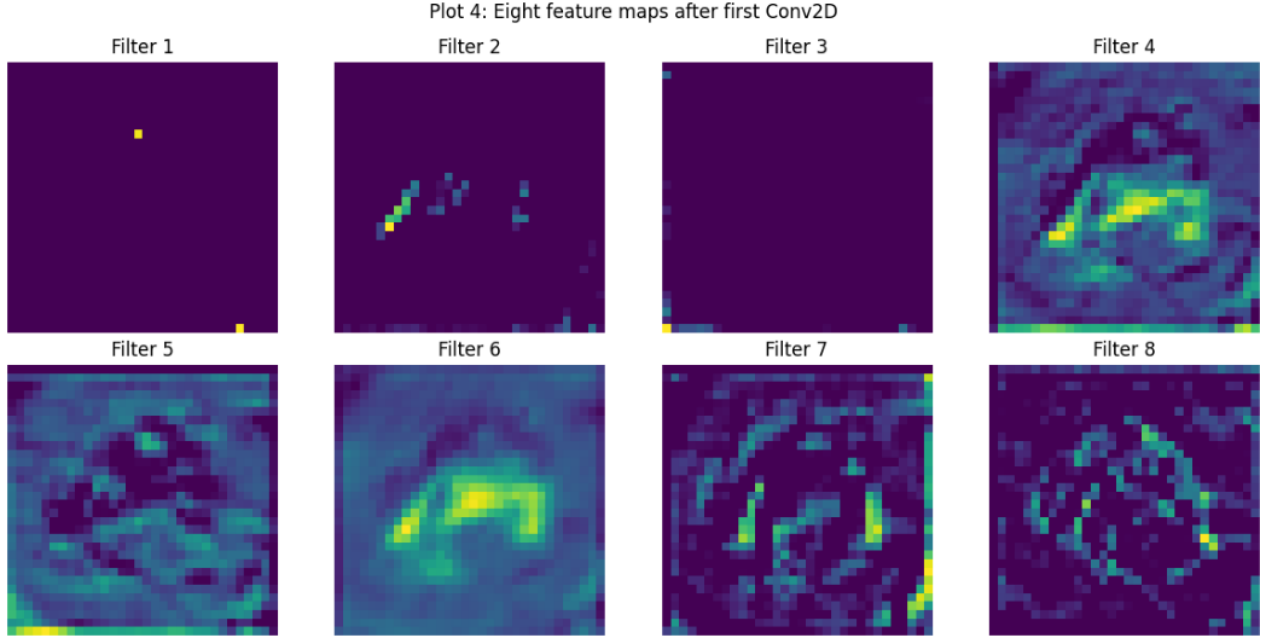


Figure 5: Eight feature maps produced by the first Conv2D layer (8 filters, 3×3 , ReLU).

Inference: Each of the 8 filters learns a distinct response: some highlight edges, some respond to colour blobs, others to smooth regions. This demonstrates that even a single convolutional layer already extracts a diverse bank of low-level features. Deeper layers would compose these into higher-level, class-discriminative representations.

Plot 6: Max vs Average Pooling (Task 5)

Task 5 requires comparing *both* the output size and the accuracy of max pooling and average pooling. The output size is computed with the same formula used for convolution,

$$N_{\text{out}} = \left\lfloor \frac{N - F + 2P}{S} \right\rfloor + 1,$$

with pool size 2×2 , stride 2, and no padding (valid).

Pooling	Pool Size	Stride	Output Size	Best Val. Accuracy
Max Pooling	2×2	2	16×16	≈ 0.687
Average Pooling	2×2	2	16×16	≈ 0.691

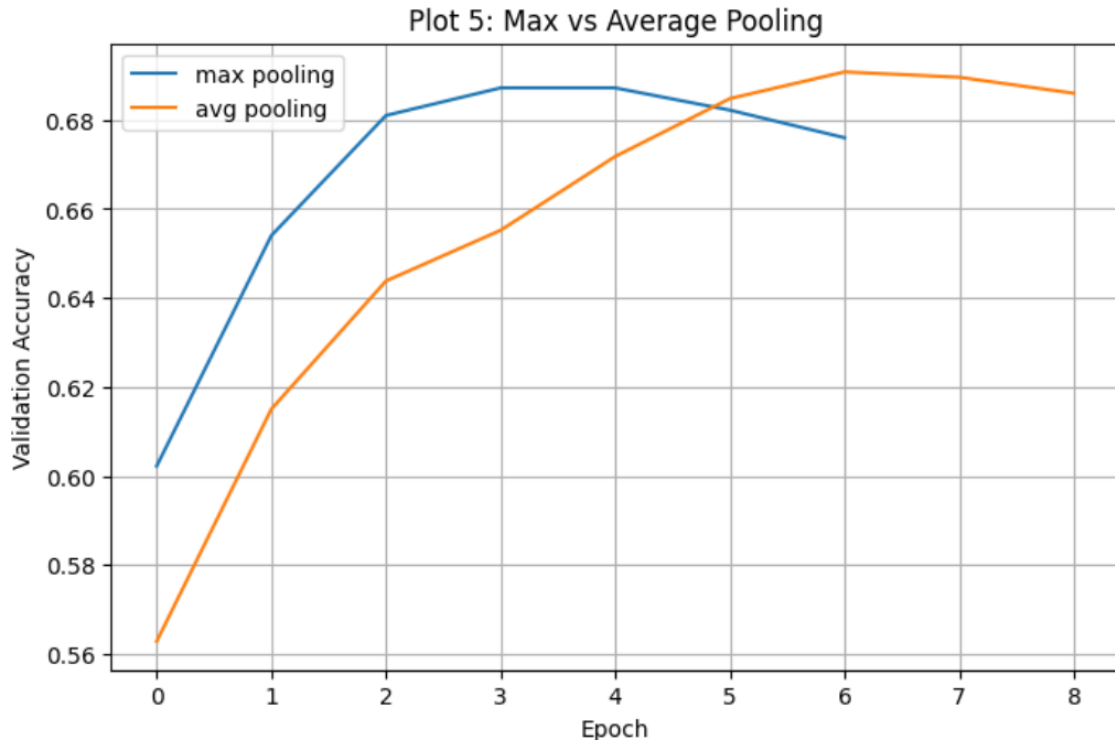


Figure 6: Validation accuracy of two identical CNNs using Max vs. Average pooling.

Inference: Both pooling methods reduce the spatial dimensions from 32×32 to 16×16 when a 2×2 pool with stride 2 is used. Average pooling achieves slightly higher final validation accuracy (≈ 0.691 vs. ≈ 0.687), while max pooling converges faster (best reached by epoch 4–5). Max pooling is generally preferred for classification because it preserves the strongest activations in each window, but average pooling is competitive and can be superior on certain texture-heavy datasets.

Plot 7: Training vs Validation Accuracy

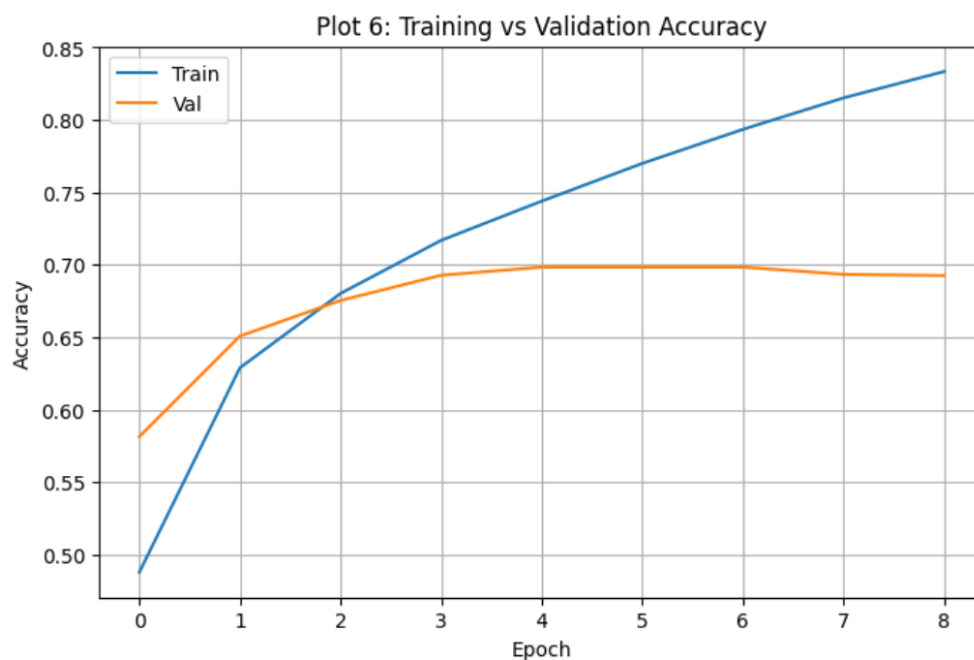


Figure 7: Training and validation accuracy. Maximum training budget = 20 epochs; early stopping (patience = 5, restore best weights) terminated training at epoch 9.

Inference: Training accuracy increases steadily to ≈ 0.83 , but validation accuracy plateaus near 0.70 from epoch 5 onward. The widening gap is a classic sign of overfitting. The model was configured to run for a maximum of 20 epochs; early stopping with patience 5 was enabled, and training automatically terminated at epoch 9 once validation accuracy stopped improving. Adding dropout or data augmentation would further help close the train-val gap.

Plot 8: Training vs Validation Loss

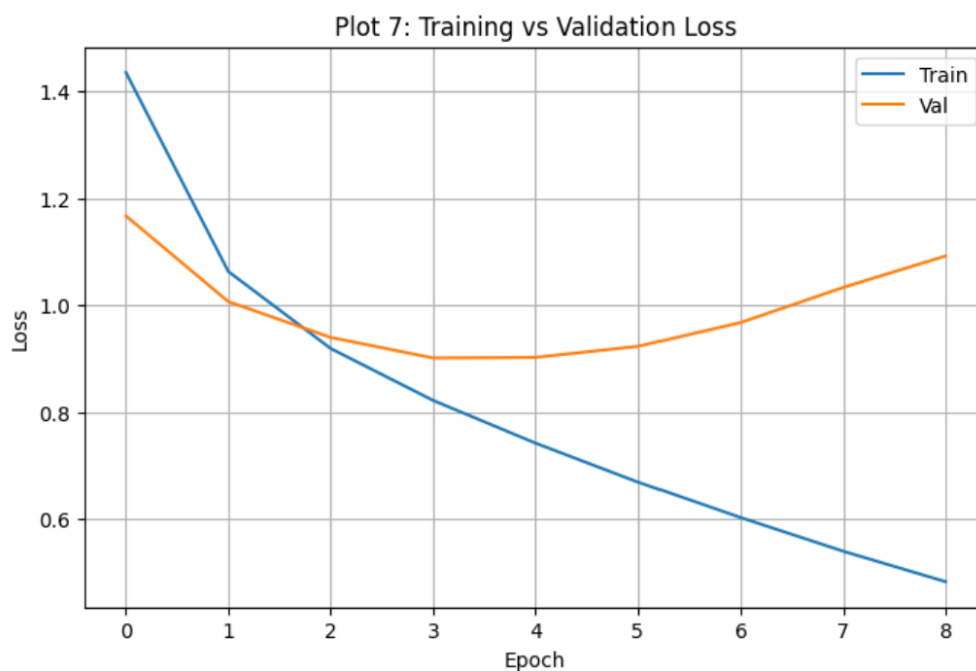


Figure 8: Training and validation loss. Same run as Plot 7: maximum 20 epochs, early stopping terminated at epoch 9.

Inference: Training loss monotonically decreases to ≈ 0.48 , while validation loss bottoms out at ≈ 0.90 around epoch 4 and then slowly rises. This divergence is consistent with the accuracy plot and confirms that the model begins to memorise the training set after a few epochs. Early stopping correctly retained the best weights (lowest validation loss) rather than the final-epoch weights.

Plot 9: Confusion Matrix

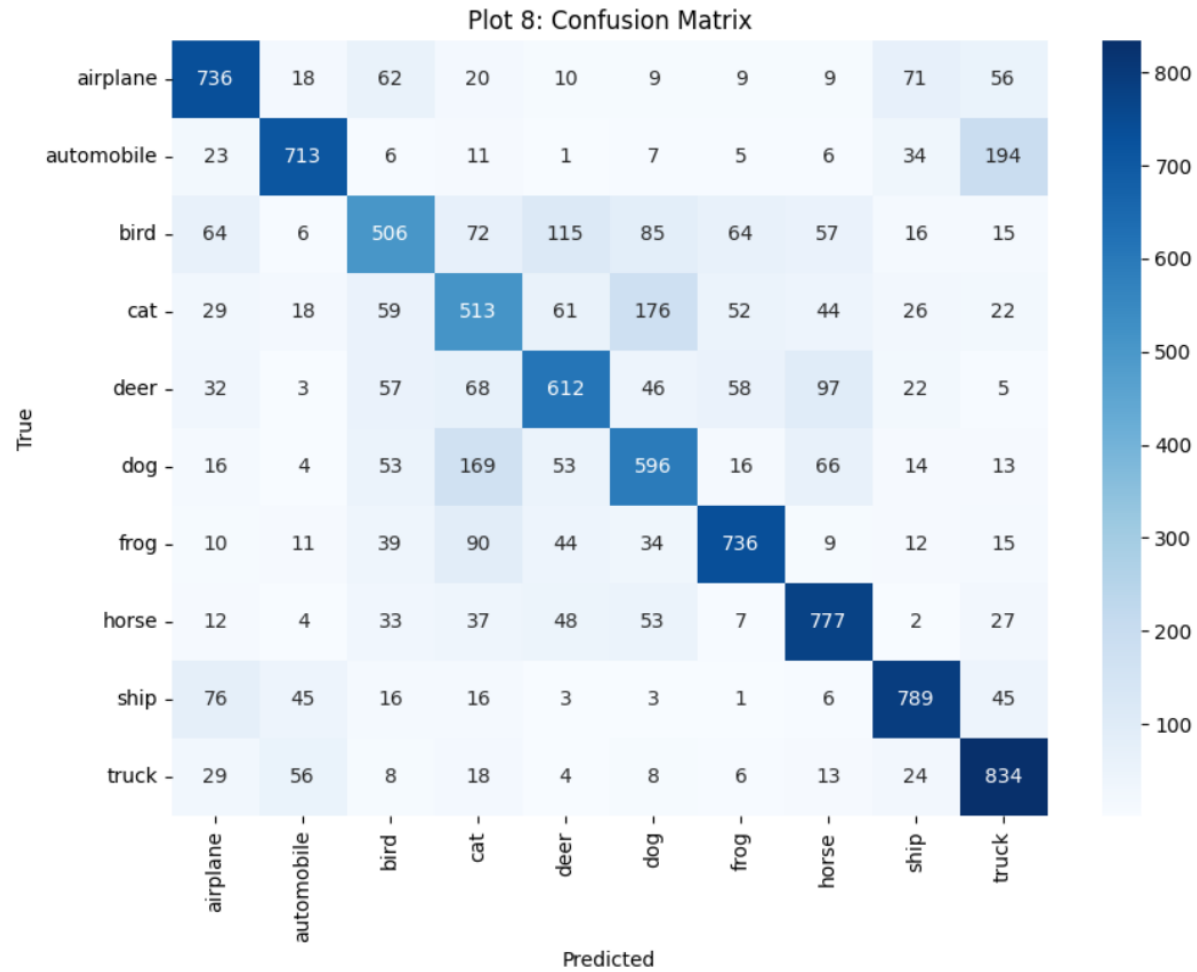


Figure 9: Confusion matrix of the trained CNN on the 10,000-image CIFAR-10 test set.

Inference: The matrix is strongly diagonal, confirming good per-class performance. The largest confusions are $cat \leftrightarrow dog$ and $deer \leftrightarrow horse$, which is expected given their visual similarity at low resolution. *Truck*, *ship*, and *automobile* are the most well-separated classes.

Plot 10: ReLU vs Sigmoid Activation

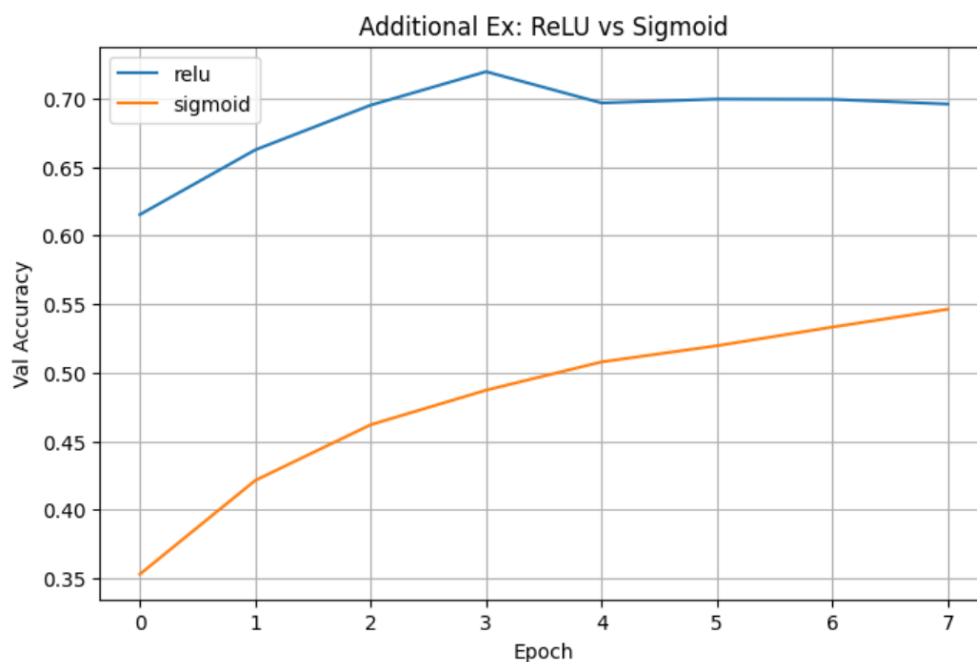


Figure 10: Validation accuracy of identical CNNs using ReLU vs. Sigmoid activations.

Inference: The ReLU network converges faster and reaches a higher validation accuracy (≈ 0.71) than the Sigmoid network (≈ 0.62). ReLU avoids the vanishing-gradient problem that plagues Sigmoid in deep networks, which is why it is the default activation for hidden layers.

Plot 11: Effect of Filter Count

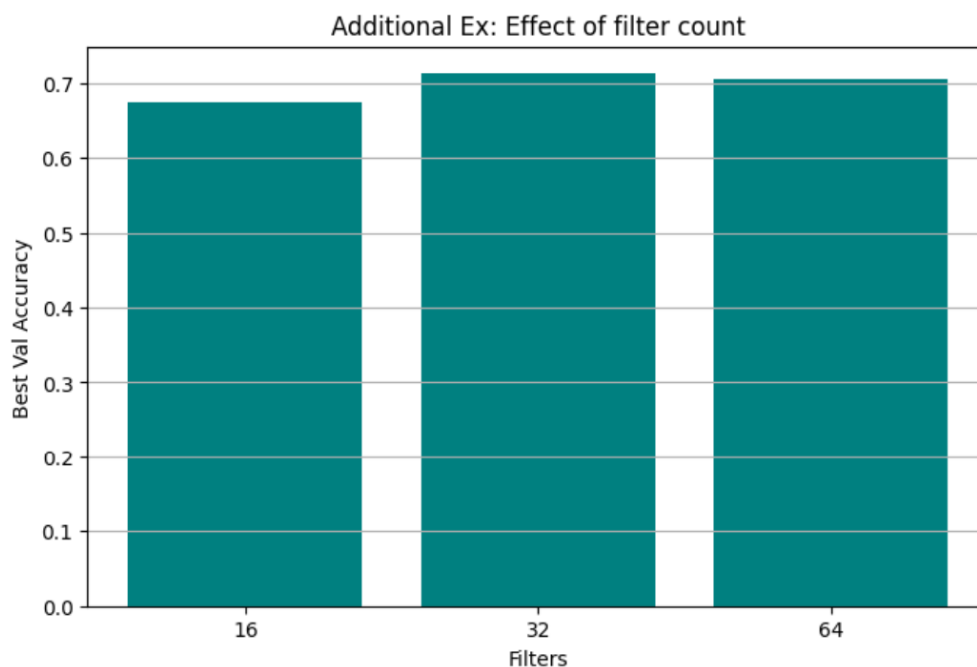


Figure 11: Best validation accuracy vs. number of filters in the first Conv2D layer.

Inference:

filters=16: best val acc=0.6744, params=268,650
filters=32: best val acc=0.7132, params=545,098
filters=64: best val acc=0.7052, params=1,125,642

Going from 16 to 32 filters gives a large accuracy gain (+3.9%) with only $2\times$ the parameters. Going from 32 to 64 filters slightly *reduces* accuracy while doubling parameters again, indicating the model has started to overfit.

Plot 12: All 10 Kernels Applied to One CIFAR-10 Image

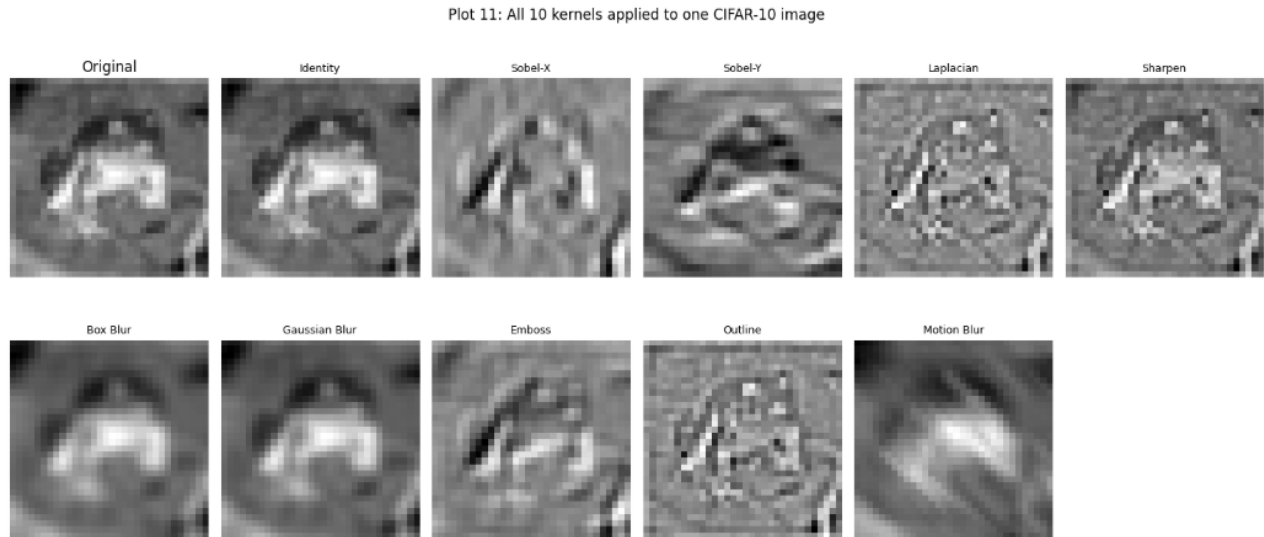


Figure 12: The ten classical kernels from Section 3.1 applied to a single grayscale CIFAR-10 image.

Inference: Identity preserves the image; Sobel-X/Y extract vertical/horizontal edges; Laplacian and Outline highlight all-direction boundaries; Sharpen enhances fine detail; Box/Gaussian Blur smooth noise; Emboss gives a 3-D relief effect; Motion Blur stretches the image diagonally. This visual set confirms the theoretical roles described in Section 3.1.

Split Plots: Training/Validation Accuracy and Loss

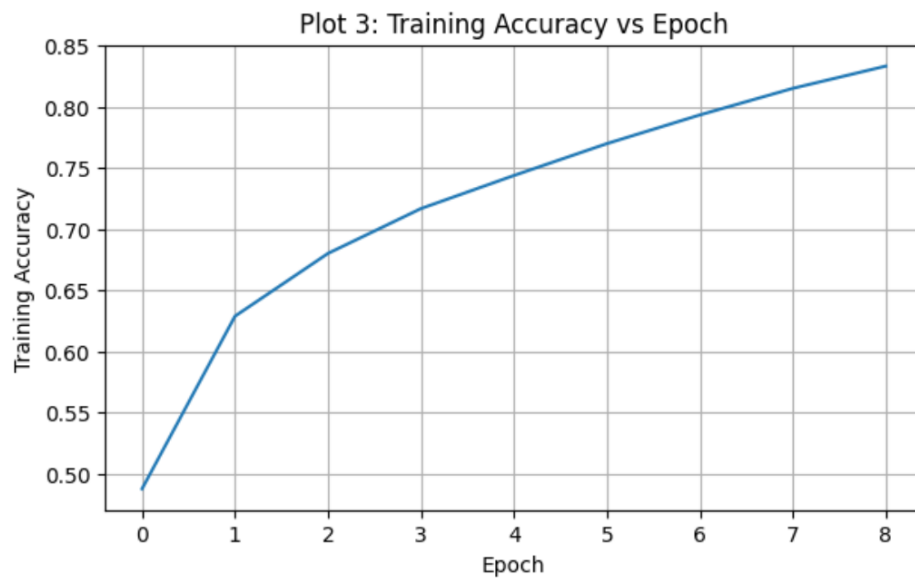


Figure 13: Training accuracy vs. epoch (isolated).

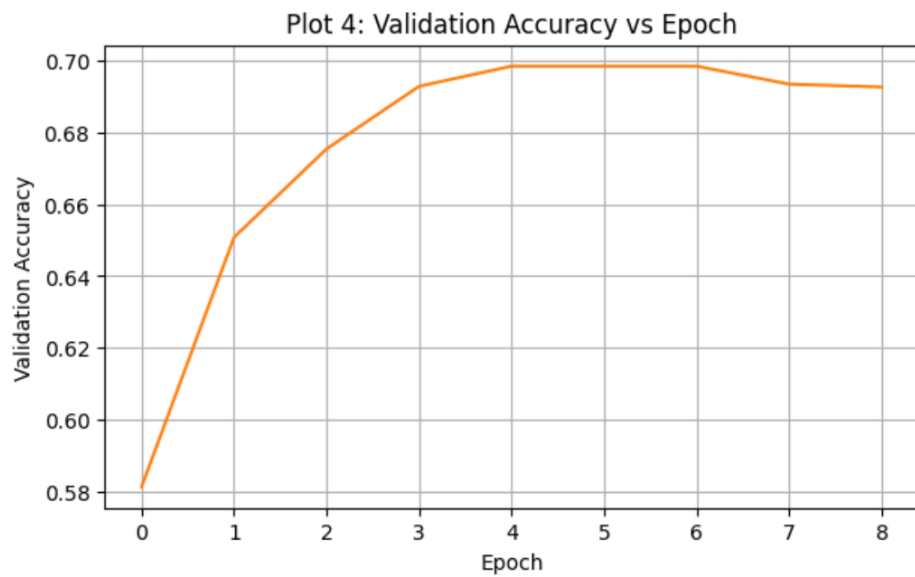


Figure 14: Validation accuracy vs. epoch (isolated).

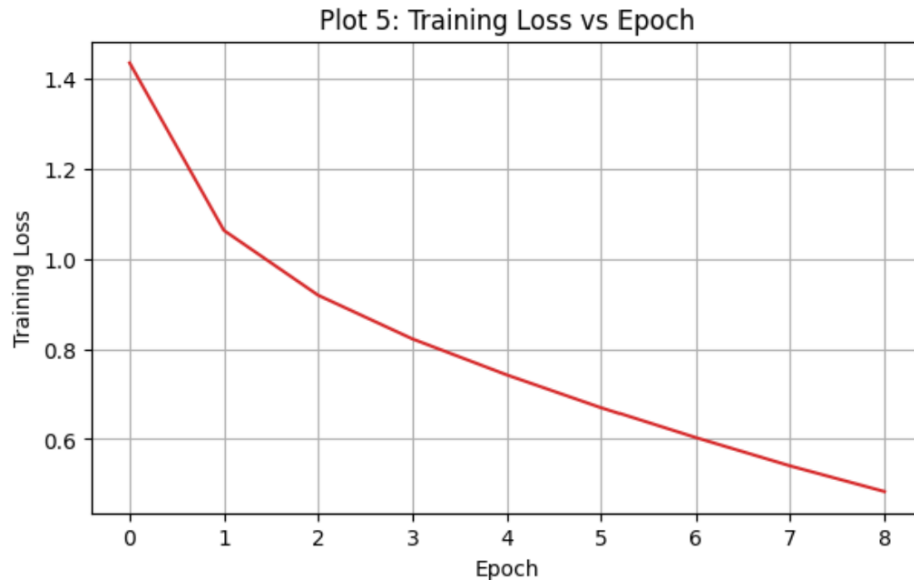


Figure 15: Training loss vs. epoch (isolated).

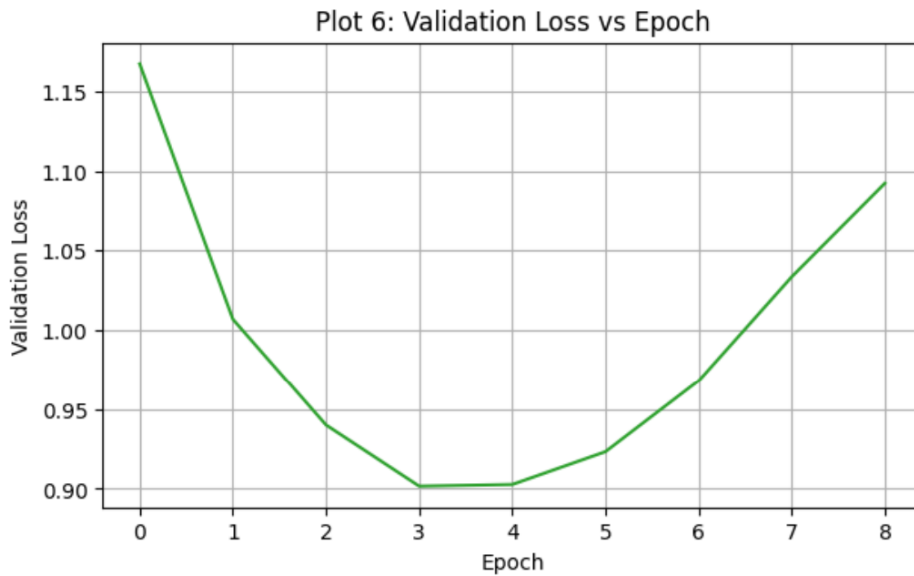


Figure 16: Validation loss vs. epoch (isolated).

Inference: The training curves are smooth and monotonic, while the validation curves flatten early (accuracy) or rise (loss), confirming that the network fits the training distribution well but generalises only up to a point before overfitting begins.

10. Additional Exercises

1. Calculate the output size for a 64×64 image using a 5×5 kernel with stride 2 and padding 2.
2. Compute the trainable parameters of a convolution layer having 64 filters of size 3×3 and an RGB input.
3. Compare ReLU and Sigmoid activation functions.
4. Replace Max Pooling with Average Pooling and compare the results.

5. Increase the number of convolution filters from 16 to 64 and analyze the effect on accuracy and computation time.

Exercise 1

$$N_{\text{out}} = \frac{64 + 2(2) - 5}{2} + 1 = \frac{63}{2} + 1 = 31 + 1 = 32$$

Ex 1: 64x64, K=5, S=2, P=2 -> 32

Exercise 2

$$\text{Params} = (3 \times 3 \times 3 + 1) \times 64 = 28 \times 64 = 1792$$

Ex 2: (3*3*3+1)*64 = 1792

Exercise 3

- **ReLU** ($f(x) = \max(0, x)$): non-saturating, sparse activation, fast convergence, avoids vanishing gradients.
- **Sigmoid** ($\sigma(x) = 1/(1+e^{-x})$): smooth, bounded in $(0, 1)$, but saturates, causes vanishing gradients in deep nets, and its output is not zero-centred.

From Plot 10, the ReLU network outperforms the Sigmoid network by $\sim 9\%$ validation accuracy on CIFAR-10.

Exercise 4

See Plot 6. Both Max and Average pooling with a 2×2 pool and stride 2 reduce the feature map from 32×32 to 16×16 . Max pooling converges faster (best ≈ 0.687 , ~ 60 s) while Average pooling reaches a slightly higher final accuracy (≈ 0.691 , ~ 55 s). Overall, max pooling is preferred for classification because it captures the most salient feature in each window and adds a small amount of translation invariance.

Exercise 5

See Plot 11. Increasing filters from $16 \rightarrow 32$ raised best validation accuracy from $0.6744 \rightarrow 0.7132$ at the cost of $2\times$ parameters ($268,650 \rightarrow 545,098$). Going from $32 \rightarrow 64$ raised parameters to 1,125,642 but *decreased* best validation accuracy to 0.7052, indicating diminishing returns and the onset of overfitting. Training time scaled roughly linearly with filter count.

11. Results

Record

- Training Accuracy
- Testing Accuracy
- Precision
- Recall
- F1-score
- Number of Parameters

Recorded Results

Metric	Value
Final Training Accuracy	≈ 0.8334 (epoch 9, early-stopped from a 20-epoch budget)
Best Validation Accuracy	≈ 0.6984 (epoch 5)
Test Accuracy	0.6812
Weighted Precision	0.6817
Weighted Recall	0.6812
Weighted F1-score	0.6797
Training time (early-stopped at epoch 9 of 20)	≈ 59.2 s
Trainable Parameters (32 filters)	545,098

Classification Report (per-class)

	precision	recall	f1-score	support
airplane	0.72	0.74	0.73	1000
automobile	0.81	0.71	0.76	1000
bird	0.60	0.51	0.55	1000
cat	0.51	0.51	0.51	1000
deer	0.64	0.61	0.63	1000
dog	0.59	0.60	0.59	1000
frog	0.77	0.74	0.75	1000
horse	0.72	0.78	0.75	1000
ship	0.78	0.79	0.79	1000
truck	0.68	0.83	0.75	1000
accuracy			0.68	10000
macro avg	0.68	0.68	0.68	10000
weighted avg	0.68	0.68	0.68	10000

Observation: Classes with distinctive shapes (*ship*, *automobile*, *frog*) score above 0.75 F1, while *cat* and *bird* score below 0.56, mirroring the confusion patterns in the confusion matrix.

12. Discussion

Answer the following.

1. Why is convolution preferred over fully connected layers for images?
2. How does stride affect the feature map size?
3. What is the role of padding?
4. Why is pooling used?
5. How do feature maps represent image characteristics?
6. Why do CNNs require fewer parameters than MLPs?

Answers

(1) Why is convolution preferred over fully connected layers for images?

A fully connected (dense) layer on a $32 \times 32 \times 3$ image already needs $3072 \times N$ weights for N units, and scales explosively for larger images. Convolution exploits three key image properties: *locality* (nearby pixels are correlated), *translation invariance* (a cat is a cat regardless of position), and *weight sharing* (the same edge detector is reused everywhere). This yields far fewer parameters, preserves spatial structure, and lets the network learn hierarchical features from pixels $\rightarrow$ edges $\rightarrow$ textures $\rightarrow$ objects.

(2) How does stride affect the feature map size?

Stride S is the step size of the kernel slide. Output size is $\lfloor (N - F + 2P)/S \rfloor + 1$. Larger stride $\Rightarrow$ fewer windows $\Rightarrow$ smaller feature map $\Rightarrow$ stronger downsampling and reduced computation. Stride 1 preserves most spatial information; stride 2 halves the spatial dimensions (common for efficient downsampling). This is experimentally confirmed in Plot 4 (Task 3): stride 2 reduces 32×32 to 16×16 (same padding) or 15×15 (valid padding).

(3) What is the role of padding?

Padding adds a border of zeros (or other values) around the input before convolution. It (a) preserves the spatial size when using `padding='same'`, (b) allows filters to process pixels at the image edges more thoroughly, and (c) provides explicit control over output dimensions via $(N - F + 2P)/S + 1$. Valid padding (no padding) shrinks the feature map by $F - 1$ pixels in each dimension, as shown in the Task 3 comparison.

(4) Why is pooling used?

Pooling (max or average) (a) reduces the spatial resolution, cutting computation and memory in deeper layers, (b) introduces local translation invariance so the network is less sensitive to small shifts, (c) increases the effective receptive field of deeper layers, and (d) acts as a mild regulariser. Max pooling preserves the strongest activation; average pooling preserves overall intensity. As shown in Plot 6, both reduce 32×32 to 16×16 for a 2×2 pool with stride 2.

(5) How do feature maps represent image characteristics?

Each filter produces a feature map whose high-value locations mark where the filter's pattern was detected. Early filters detect simple patterns (edges, blobs, colours) – as seen in Plot 5, where 8 filters gave 8 distinct responses. Deeper filters combine these into complex concepts (eyes, wheels, fur). Thus a feature map is a spatial “confidence heat-map” of a learned pattern detector.

(6) Why do CNNs require fewer parameters than MLPs?

Three reasons: (a) *local connectivity* – each neuron connects only to a small $F \times F$ patch, not the whole image; (b) *weight sharing* – the same kernel is reused across all spatial positions, so a 3×3 filter has only 9 weights (plus bias) regardless of image size; (c) *hierarchical composition* – depth is built from stacking small filters rather than from ever-larger dense matrices. For our network, the first Conv2D layer has only 896 parameters versus 3 072 000+ for an equivalent dense layer on $32 \times 32 \times 3$.

13. References

1. Goodfellow et al., *Deep Learning*.
2. Bishop, *Pattern Recognition and Machine Learning*.
3. Haykin, *Neural Networks and Learning Machines*.
4. TensorFlow Documentation.
5. CIFAR-10 Dataset Documentation.